



CampusLab

Arquitectura Cloud Native segura — Caso 2

Federación Microsoft Entra ID + Amazon Cognito · API Gateway · BFF · EC2/Docker

Katherine Ramírez · Marcelo Prado · Williams Rivas

Ingeniería en Informática — DSY1107 Desarrollo Cloud Native I · Septiembre 2026

— Agenda

Recorrido de la solución: de la identidad al despliegue

- ◆ Contexto y objetivo
- ◆ Alcance del entregable
- ◆ Arquitectura del flujo seguro
- ◆ Identidad federada (Entra + Cognito)
- ◆ Login OIDC + PKCE
- ◆ Evidencia real del login
- ◆ API Gateway: validación en el borde
- ◆ Autorización por rol (BFF)
- ◆ Evidencia del CRUD real
- ◆ Validación de esquema y estados
- ◆ Despliegue en EC2 con Docker
- ◆ Conclusiones y repositorio

— Contexto y objetivo

- ◆ **CampusLab:** reserva de laboratorios y equipos académicos para una red de instituciones.
- ◆ **Objetivo:** sistema funcional y seguro en la nube, con identidad corporativa de Microsoft.
- ◆ Stack: **React** · **Spring Boot** · **Cognito** federado a Entra · **API Gateway** · **EC2/Docker**.
- ◆ Regla de oro: JWT → API Gateway → BFF → microservicio de dominio.

— Alcance del entregable

Decisiones conscientes del Caso 2

✓ Incluido en el core

- ◆ Identidad federada (Entra + Cognito)
- ◆ API Gateway único como puerta
- ◆ BFF de seguridad (roles + CORS)
- ◆ Microservicios catalog y bookings
- ◆ Despliegue en Docker / EC2

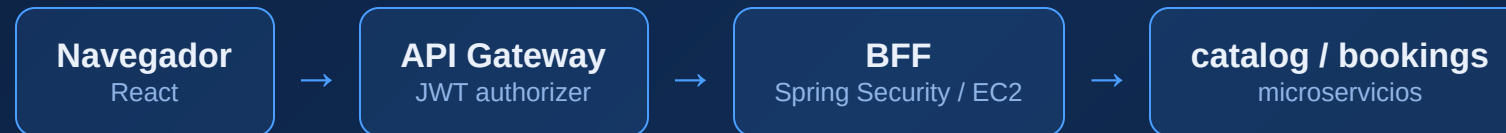
✗ Fuera de alcance

- ◆ Mensajería asíncrona (Kafka)
- ◆ Métricas, KPIs y analítica
- ◆ Auditoría por streaming
- ◆ AWS Lambda (integración HTTP directa)

— Arquitectura — flujo seguro

Componentes y recorrido de la petición

- ◆ Frontend **React** (MSAL + Cognito PKCE)
- ◆ Amazon **Cognito** (IDaaS, federado a Entra)
- ◆ API Gateway HTTP API (**JWT authorizer**)
- ◆ BFF **Spring Security** (roles, CORS)
- ◆ Microservicios **catalog** + **bookings**



— Identidad federada — Entra ID + Cognito

- ◆ **App Registration** en Entra ID (tenant WRIVAST).
- ◆ **Cognito**: proveedor OIDC → EntraID; grupos **Admin** / **Operador**.
- ◆ App Client **público** con Authorization Code + PKCE.
- ◆ El login se **delega en Microsoft**; Cognito emite el token.

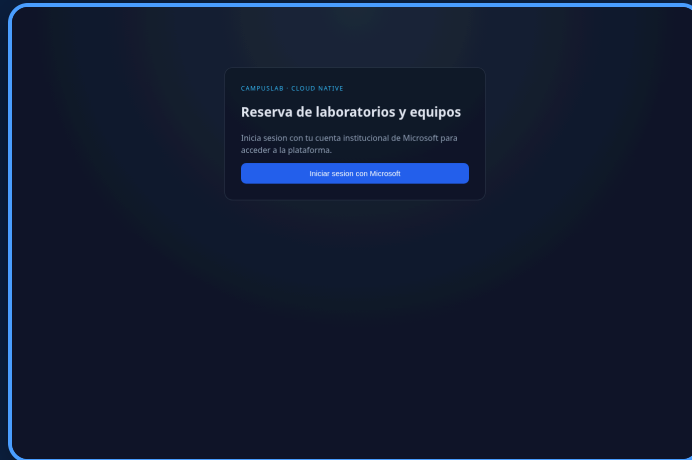
```
User Pool : us-east-1_dB0QeniQq (dominio campuslab-wrivast)
App Client : 6r3af8ucc96okla0ujph9pru4a (PKCE, sin secreto)
Federación real: login.microsoftonline.com
```

— Login OIDC — Authorization Code + PKCE

- ◆ El cliente genera `code_verifier` y `code_challenge` (S256).
- ◆ App Client público: `sin secreto` en el navegador.
- ◆ Token con `token_use=access`, `client_id` y `cognito:groups`.
- ◆ El frontend adjunta `Authorization: Bearer` en cada llamada.

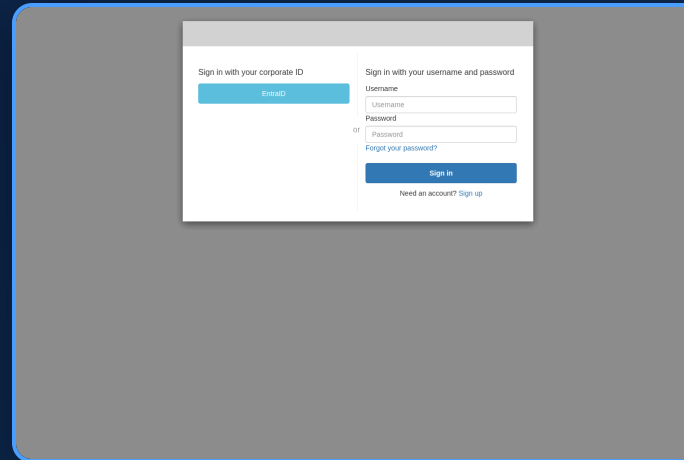
Evidencia real del login federado

Capturas del entorno desplegado · flujo OIDC + PKCE



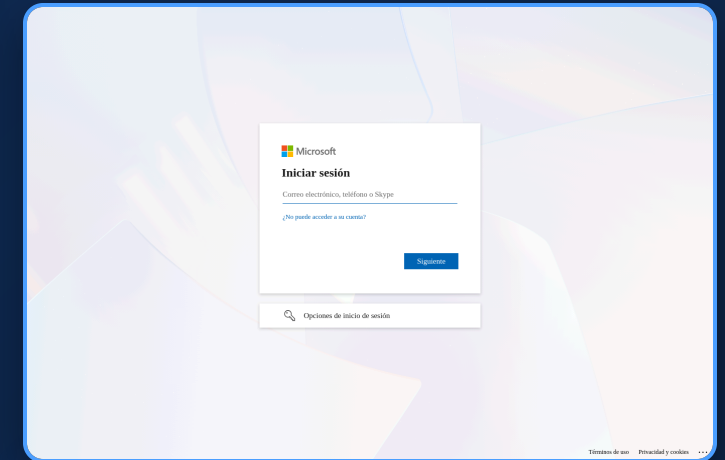
1

Frontend CampusLab
Botón “Iniciar sesión con Microsoft”



2

Hosted UI de Cognito
Proveedor federado EntraID (OIDC)



3

Login de Microsoft
Entra ID valida la identidad real

— API Gateway — validación en el borde

La primera frontera de seguridad

- ◆ **JWT Authorizer:** valida Issuer y Audience (client_id) automáticamente.
- ◆ **CORS:** restringido al origen del frontend (localhost:5173).
- ◆ Petición **sin token** → **401**; con token válido → **200**.

```
Issuer    : cognito-idp.us-east-1.amazonaws.com/us-east-1_dB0QeniQq
Audience : 6r3af8ucc96okla0ujph9pru4a
GET /api/catalog/resources (sin token) -> 401 Unauthorized
```

— Autorización por rol — BFF

El segundo control

- ◆ El API Gateway filtra autenticación; el **BFF aplica autorización granular por endpoint**.
- ◆ `cognito:groups` → **ROLE_ADMIN** / **ROLE_OPERADOR** (Spring Security).
- ◆ Usuario sin rol Admin en un endpoint protegido → **403 Forbidden**.

```
POST /api/catalog/resources    Usuario estándar (sin rol Admin)
Resultado: rechazado por Spring Security -> HTTP/1.1 403 Forbidden
```

— Evidencia del CRUD real

Contra el Invoke URL del API Gateway, con tokens reales de Cognito

Método	Endpoint	Rol	Código real
GET	/api/catalog/resources	sin token	401 Unauthorized
GET	/api/catalog/resources	Admin	200 OK (5 productos)
POST	/api/catalog/resources	Admin	201 Created
POST	/api/catalog/resources	usuario no Admin	403 Forbidden
POST	/api/bookings	Admin	201 (SOLICITADA)
PUT	/api/bookings/2/status	Admin	200 (EN_USO)
DELETE	/api/bookings/1	Admin	204 No Content

— Validación de esquema y estados de la reserva

Validación estricta (400)

- ◆ Rechazo inmediato **antes** de tocar la BD.
- ◆ Detalle del error **por campo** (nombre, rut).
- ◆ **HTTP 400** con formato de rut requerido.

Máquina de estados



- ◆ No se pasa a EN_USO sin APROBADA.
- ◆ Transiciones limitadas a Admin/Operador.

— Despliegue en EC2 con Docker

Contenedores en Amazon EC2

- ◆ EC2 Amazon Linux 2023 (t3.medium) — **i-0448753bab6e8173b** — **3.237.97.7**.
- ◆ Docker Compose: **catalog**, **bookings** y **BFF**.
- ◆ Security Group: solo puertos **22**, **8080** y **8090**.

```
$ sudo docker compose ps
campuslab-catalog   Up      8081
campuslab-bookings  Up      8082
campuslab-bff       Up      0.0.0.0:8080->8080
$ curl http://3.237.97.7:8080/actuator/health -> {"status":"UP"}
```

— Conclusiones y alcance

- ◆ Arquitectura Cloud Native **funcional y segura**, desplegada de extremo a extremo.
- ◆ Identidad **federada a Microsoft**; JWT validado en API Gateway y BFF; roles y CORS.
- ◆ CRUD con validación de esquema (nombre/rut/fecha): **200 / 201 / 204 / 400 / 401 / 403**.
- ◆ Fuera de alcance: Kafka, KPIs/analytics, auditoría por streaming y AWS Lambda.
- ◆ Repositorio: github.com/WilliamsRT93/campuslab-caso2